

Università degli Studi di Napoli Federico II

Corso di Laurea Magistrale in Ingegneria Informatica

Corso di Software Architecture Design

Prof. S. Fasolino - A.A. 2025 - 26

Progetto

TESTING ROBOT CHALLENGE

Gruppo di Progetto D14

Studenti	Matricola	E-mail
Pietro Moccia	DE9000174	pie.moccia@studenti.unina.it
Silvia Pedata	DE9000163	sil.pedata@studenti.unina.it
Ciro Musone	DE9000162	cir.musone@studenti.unina.it

Versione di partenza

[GitHub - Testing-Game-SAD-2023/A13: Versione migliorativa sviluppata a partire dal progetto A10-2024 con integrazione del repository A7](#)

Consegna

[GitHub - Testing-Game-SAD-2023/A13 at D14_TaskR6_ProfiloGiocatore](#)



SOMMARIO

1. Specifiche Informali.....	3
2. Analisi e Specifica dei Requisiti.....	4
3. Progettazione.....	14
4. Implementazione.....	20
5. Testing.....	33
6. Sviluppi futuri e suggerimenti.....	39

Gestire la pagina dedicata al profilo del Giocatore, prevedendo una parte fissa (Foto/Avatar e Bio del giocatore) ed una parte variabile di informazioni da scegliersi usando un selettore sulla UI

APPROCCIO DI SVILUPPO SEGUITO

Per la progettazione e l'implementazione del modulo di gestione del profilo, il gruppo di lavoro ha adottato un approccio collaborativo e iterativo, ispirato ai principi delle metodologie Agile. Tale scelta è stata dettata dalla necessità di gestire la complessità dell'integrazione con sistemi preesistenti e di garantire un elevato standard qualitativo del software prodotto.

Il team ha operato in un ambiente di lavoro sincrono, sfruttando piattaforme di comunicazione remota (Microsoft Teams) per simulare una co-presenza fisica. Questa modalità ha permesso la condivisione costante dello schermo e delle decisioni tecniche in tempo reale, favorendo la proprietà collettiva del codice.

Sebbene ogni membro si sia focalizzato su aree specifiche quali lo sviluppo del Front-End, la gestione della biografia e delle immagini, l'integrazione del codice legacy e il sistema di messaggistica, ogni soluzione è stata sottoposta a discussione e revisione congiunta. Per le componenti di maggiore criticità tecnica, sono state applicate pratiche riconducibili al *Pair/Ensemble Programming*, permettendo una risoluzione collegiale dei problemi e una riduzione dei tempi di debug.

Lo sviluppo è progredito per incrementi successivi, con review ogni due settimane insieme alla professoressa e ai colleghi, in cui sono state mostrate le funzionalità completate per raccogliere feedback e definire gli obiettivi per l'iterazione successiva. Questo ha permesso di migliorare il lavoro, correggerne la direzione e migliorare continuamente qualità e funzionalità dell'app. Di fondamentale importanza sono stati anche gli incontri online con l'ingegnere Perlotto per risolvere dubbi e ottenere chiarimenti sull'implementazione.

2. ANALISI E SPECIFICA DEI REQUISITI

REQUISITI

1. Il sistema deve offrire al player una funzionalità per accedere alla sezione del profilo
2. Il sistema deve offrire al player una funzionalità per modificare la biografia
3. Il sistema deve offrire al player una funzionalità per modificare l'immagine del profilo
4. Il sistema deve offrire al player una funzionalità per visualizzare l'immagine, la biografia, il nome utente, l'e-mail, il livello corrente, i punti esperienza totali, i punti esperienza mancanti al livello successivo, il numero di Follower ed il numero di Following
5. Il sistema deve offrire al player la possibilità di visualizzare gli Achievements
6. Il sistema deve offrire al player la possibilità di visualizzare la Game History
7. Il sistema deve offrire al player la possibilità di visualizzare la Leaderboard
8. Il sistema deve offrire al player la possibilità di visualizzare i Team a cui si partecipa
9. Il sistema deve offrire al player la possibilità di visualizzare le notifiche
10. Il sistema deve offrire al player la possibilità di visualizzare la lista dei Following
11. Il sistema deve offrire al player la possibilità di visualizzare la lista dei Follower
12. Il sistema deve offrire al player la possibilità di visualizzare i messaggi inviati
13. Il sistema deve offrire al player la possibilità di visualizzare i messaggi ricevuti
14. Il sistema deve offrire al player la possibilità di inviare un messaggio ad un Follower o Following
15. Di ogni messaggio dell'inbox si vuole memorizzare il contenuto, il mittente, data ed ora di invio
16. Di ogni messaggio dell'outbox si vuole memorizzare il contenuto, il destinatario, data ed ora di invio
17. Il sistema deve offrire al player una funzionalità per eliminare i messaggi nell'inbox e outbox
18. Il sistema deve offrire al player una funzionalità per cercare altri player
19. Il sistema deve offrire al player una funzionalità per visualizzare il profilo di un altro player
20. Del profilo di un altro player si vogliono memorizzare il nome utente, l'e-mail, l'immagine del profilo, la biografia, il numero di Follower e Following, il livello, i punti di esperienza totali e i punti esperienza mancanti al livello successivo
21. Il sistema deve offrire al player una funzionalità di aggiungere un utente come follower
22. Il sistema deve offrire al player una funzionalità di rimuovere un utente come follower
23. La biografia contiene di default "Test addicted" per i nuovi utenti.
24. La biografia può contenere al massimo 500 caratteri.
25. Se l'immagine di profilo non viene fornita dall'utente, deve essere visualizzata un'immagine placeholder predefinita.
26. Immagine del profilo e biografia sono opzionali
27. I messaggi devono avere una lunghezza massima di 1000 caratteri.

CLASSIFICAZIONE DEI REQUISITI

Requisiti funzionali

ID	Requisito	Ordine
RF01	Il sistema deve offrire al player una funzionalità per accedere alla sezione del profilo	1
RF02	Il sistema deve offrire al player una funzionalità per modificare la biografia	2

RF03	Il sistema deve offrire al player una funzionalità per modificare l'immagine del profilo.	3
RF04	Il sistema deve offrire al player una funzionalità per visualizzare l'immagine, la biografia, il nome utente, l'e-mail, il livello corrente, i punti esperienza totali, i punti esperienza mancanti al livello successivo, il numero di Follower ed il numero di Following.	4
RF05	Il sistema deve offrire al player la possibilità di visualizzare gli Achievements	5
RF06	Il sistema deve offrire al player la possibilità di visualizzare la Game History.	6
RF07	Il sistema deve offrire al player la possibilità di visualizzare la Leaderboard.	7
RF08	Il sistema deve offrire al player la possibilità di visualizzare i Team a cui si partecipa	8
RF09	Il sistema deve offrire al player la possibilità di visualizzare le notifiche.	9
RF10	Il sistema deve offrire al player la possibilità di visualizzare la lista dei Following.	10
RF11	Il sistema deve offrire al player la possibilità di visualizzare la lista dei Follower.	11
RF12	Il sistema deve offrire al player la possibilità di visualizzare i messaggi inviati	12
RF13	Il sistema deve offrire al player la possibilità di visualizzare i messaggi ricevuti	13
RF14	Il sistema deve offrire al player la possibilità di inviare un messaggio ad un Follower o Following.	14

RF15	Il sistema deve offrire al player una funzionalità per eliminare i messaggi nell'inbox e outbox.	17
RF16	Il sistema deve offrire al player una funzionalità per cercare altri player.	18
RF17	Il sistema deve offrire al player una funzionalità per visualizzare il profilo di un altro player.	19
RF18	Il sistema deve offrire al player una funzionalità di aggiungere un utente come follower.	21
RF19	Il sistema deve offrire al player una funzionalità di rimuovere un utente come follower.	22

Requisiti sui dati

ID	Requisito	Ordine
RD01	Di ogni messaggio dell'inbox si vuole memorizzare il contenuto, il mittente, data ed ora di invio.	15
RD02	Di ogni messaggio dell'outbox si vuole memorizzare il contenuto, il destinatario, data ed ora di invio.	16
RD03	Del profilo di un altro player si vogliono memorizzare il nome utente, l'e-mail, l'immagine del profilo, la biografia, il numero di Follower e Following, il livello, i punti di esperienza totali e i punti esperienza mancanti al livello successivo.	20
RD04	La biografia contiene di default "Test addicted" per i nuovi utenti.	23
RD05	Se l'immagine di profilo non viene fornita dall'utente, deve essere visualizzata un'immagine placeholder predefinita.	25
RD06	Immagine del profilo e biografia sono opzionali	26

Vincoli

ID	Requisito (Vincolo)	Ordine
V01	La biografia può contenere al massimo 500 caratteri.	24

V02	Il nome e l'immagine del Team possono essere modificati solamente dall'admin.	28
V03	I messaggi devono avere una lunghezza massima di 1000 caratteri.	27

MODELLAZIONE DEI CASI D'USO

Il diagramma dei casi d'uso è un diagramma UML che descrive in modo sintetico cosa può fare un attore (nel nostro caso il Player) interagendo con il sistema.

Nel nostro contesto il Player può:

- Usare i casi d'uso **Follow/Unfollow**, eseguibile solo quando si visualizza il profilo di un altro utente, e **ModificaProfilo**, che consente di cambiare immagine e biografia solo quando il profilo visualizzato è il proprio, che includono il caso d'uso **VisualizzaProfilo**, che invece mostra un profilo con dati statici: immagine, biografia, nome utente, e-mail, livello corrente, punti esperienza totali, punti mancanti al livello successivo, numero di Follower e di Following (sia del proprio profilo sia di un generico utente).
- Eseguire **AccediStatistiche**, che permette di consultare le statistiche di gioco, la leaderboard, lo storico delle partite, gli achievement e di navigare verso le schede Team, Notifiche e Social. Questo caso d'uso può essere usato dal Player solo se sta visualizzando il proprio profilo.
- Usare **VisualizzaMessaggio** per vedere i messaggi in inbox e outbox e il caso d'uso **EliminaMessaggi** che include VisualizzaMessaggio, perché per poter eliminare un messaggio è necessario prima visualizzarlo nella lista.
- Usare **CercaProfilo** per cercare un altro player
- Usare **InvioMessaggio** per inviare un nuovo messaggio a un utente che è tra i propri follower o following.

Attori primari:

- Player

Casi d'uso:

- **UC1:** ModificaProfilo
- **UC2:** Follow/Unfollow
- **UC3:** AccediStatistiche
- **UC4:** CercaProfilo
- **UC5:** InvioMessaggio
- **UC6:** EliminaMessaggio

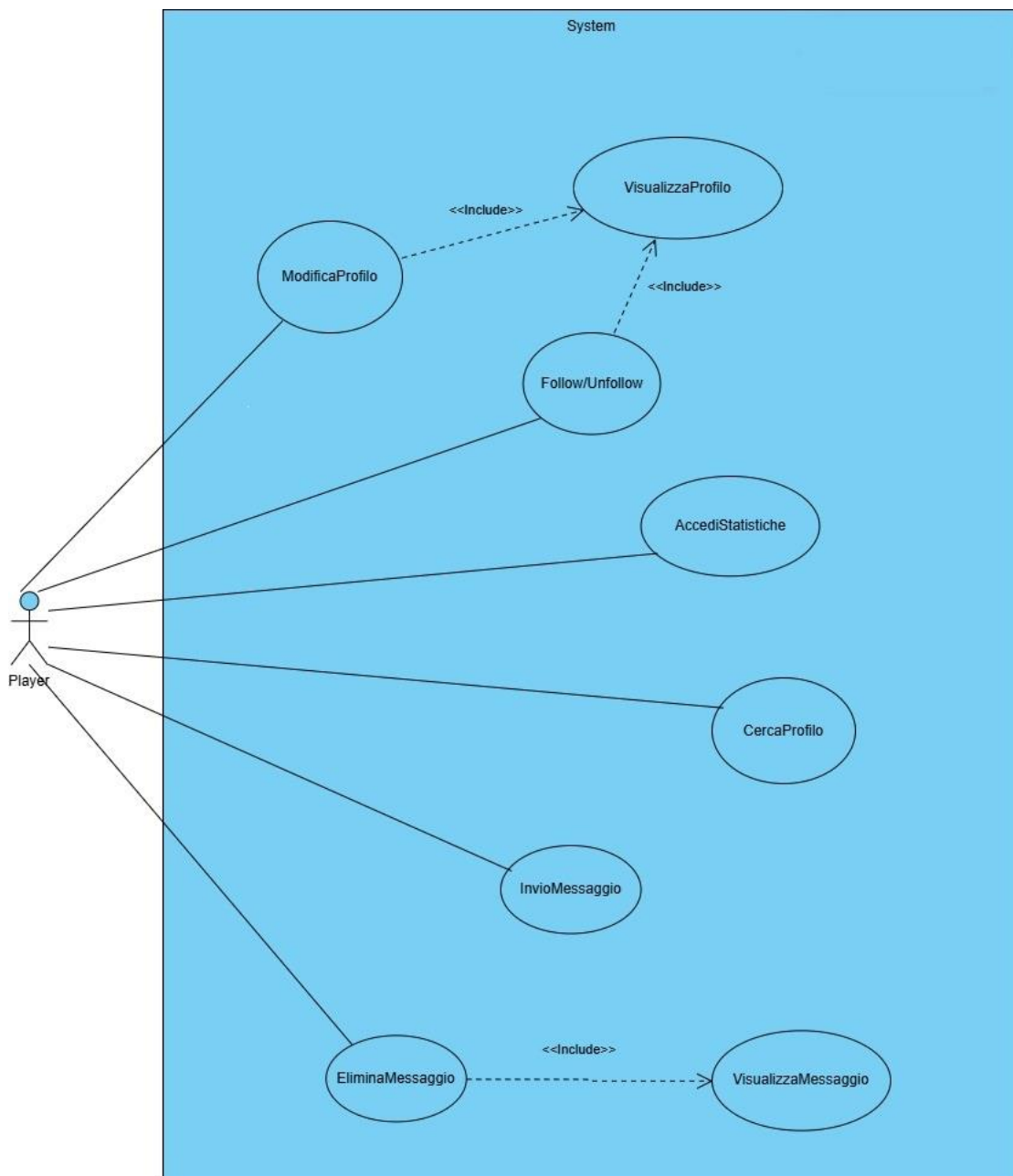
Casi d'uso di inclusione:

- **UC7:** VisualizzaProfilo
- **UC8:** VisualizzaMessaggio

Caso d'uso	Attori primari / secondari	Inclusione / Estensione	Requisiti
UC1: VisualizzaProfilo	Primario: - Secondario: -	-	RF01, RF04, RF17
UC2: ModificaProfilo	Primario: Player Secondario: -	Include: VisualizzaProfilo	RF02, RF03
UC3: AccediStatistiche	Primario: Player Secondario: -	-	RF05, RF06, RF07, RF08, RF09, RF10, RF11
UC4: VisualizzaMessaggio	Primario: - Secondario: -	-	RF12, RF13
UC5: CercaProfilo	Primario: Player Secondario: -	-	RF16
UC6: InvioMessaggio	Primario: Player Secondario: -	-	RF14
UC7: Follow/Unfollow	Primario: Player Secondario: -	Include: VisualizzaProfilo	RF18, RF19
UC8: EliminaMessaggio	Primario: Player Secondario: -	Include: VisualizzaMessaggio	RF15

DIAGRAMMA DEI CASI D'USO

Si presenta il diagramma dei casi d'uso relativo alla trattazione precedente:



SCENARI

Prendiamo in esame due scenari relativi alle funzionalità realizzate:

Caso d'uso	ModificaProfilo
ID	UC2
Breve Descrizione	Il player, dopo aver inserito la nuova biografia e aver selezionato la nuova immagine del profilo, le salva ed il suo profilo viene aggiornato.
Attori Primari	Player
Attori Secondari	-
Precondizioni	1. Il player deve essere autenticato nel sistema. 2. Il player deve essere nella sua pagina di profilo.
Sequenza degli eventi principale	1. Il caso d'uso inizia quando il player è entrato nella pagina di modifica del profilo. 2. Il player inserisce la biografia. 3. Il player seleziona l'immagine di profilo desiderata. 4. Il player clicca sul pulsante "Salva profilo". 5. Il sistema restituisce un messaggio di conferma.
Post-Condizioni	Il profilo del player è aggiornato con le nuove scelte.
Sequenza degli eventi alternativa	A1: Se il player non riempie le caselle del form, saranno salvati i valori di default o quelli precedentemente inseriti. A2: Se il player inserisce più di 500 caratteri riceve un messaggio di errore.

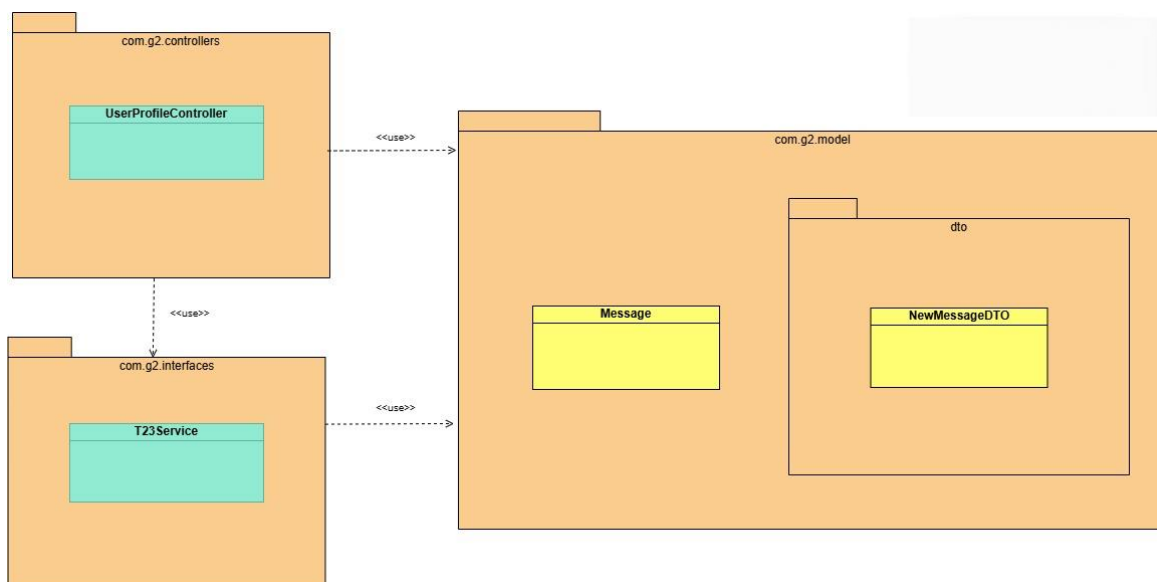
Caso d'uso	EliminaMessaggio
ID	UC8
Breve Descrizione	Il player, dopo aver visualizzato il messaggio nella sezione di inbox o outbox, lo elimina.
Attori Primari	Player
Attori Secondari	-
Precondizioni	1. Il player deve essere autenticato nel sistema.

	2. Il player deve essere nella sezione dei messaggi di inbox o outbox. 3. Il player deve essere o il mittente o il destinatario del messaggio che desidera eliminare.
Sequenza degli eventi principale	1. Il caso d'uso inizia quando il player visualizza i messaggi. 2. Il player seleziona il messaggio da eliminare con il pulsante "Delete". 3. Il player conferma di voler eliminare il messaggio.
Post-Condizioni	Il messaggio eliminato dal player verrà rimosso sia per il mittente che per il destinatario.
Sequenza degli eventi alternativa	A1: Se il player non conferma l'eliminazione del messaggio cliccando "Annulla" il messaggio non sarà rimosso.

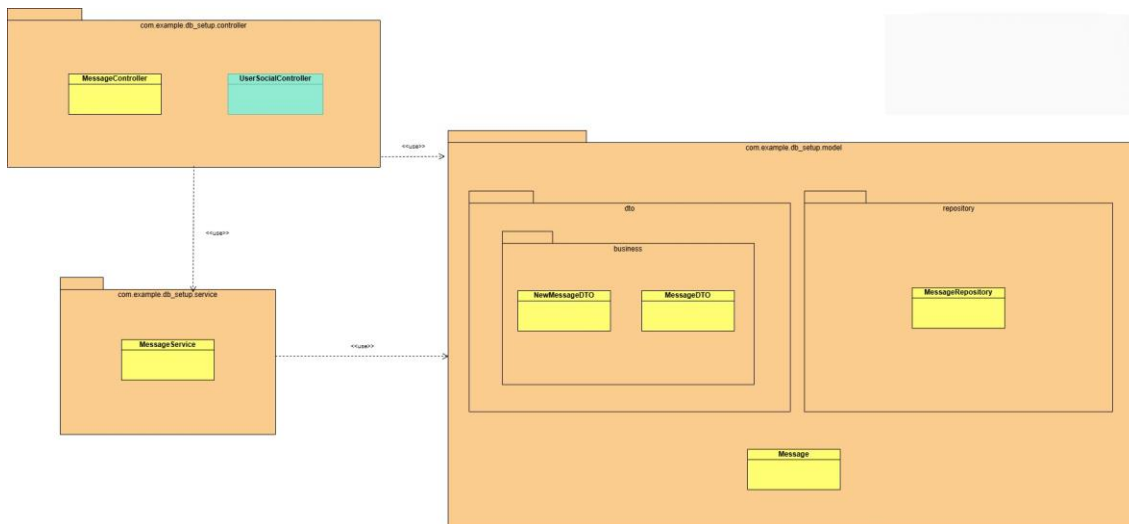
DIAGRAMMA DEI PACKAGE

Si mostrano i diagrammi dei package dei microservizi impattati dal lavoro svolto dal team D14, ponendo particolare enfasi sui package e classi di interesse, aggiunti e/o modificati durante il lavoro. Infatti, per non appesantire la rappresentazione, si è deciso di trascurare package e classi non modificate e non coinvolte nello sviluppo. Si può dedurre l'impatto su ciascun package/classe osservando i colori nella Legenda.

- MICROSERVIZIO T5



- MICROSERVIZIO T2



Legenda:

IMMUTATO

MODIFICATO

NUOVO

ANALISI D'IMPATTO

Con lo scopo di perseguire gli obiettivi specificati all'interno del task assegnato, si è deciso di lavorare su una grande varietà di componenti del sistema. Per quanto riguarda quelli di Front-end, sono stati modificati i seguenti file appartenenti al microservizio T5:

- *Profile.js*
- *Edit-profile.html*
- *Profile.html*
- *Profile-edit.js*

Questi file sono impattanti sull'adeguamento dell'interfaccia e della logica di modifica/visualizzazione del profilo utente, statistiche, informazioni social e messaggi.

Contemporaneamente sono stati modificati ed introdotto le seguenti classi di Back-end nel microservizio T5:

- *UserProfileController.java*
- *T23Service.java*

sono state impattate per la gestione delle nuove/aggiornate operazioni sul profilo e per l'interazione con il microservizio T23. Le seguenti sono state create per gestire la messagistica:

- *NewMessageDTO.java*: è il DTO che T5 usa per inviare a T23 i dati di un nuovo messaggio da creare.
- *Message.java*: rappresenta il messaggio che T5 riceve da T23 e mostra nella UI

Mentre per quanto riguarda il microservizio T23:

- *UserSocialController.java*

per supportare le nuove richieste provenienti dal microservizio T5 e l'aggiornamento/recupero delle informazioni social del giocatore. Per la gestione dei messaggi, in questo microservizio sono state aggiunte le seguenti classi:

- *Message.java*: modella la riga della tabella Messages nel database

- *MessageDTO.java*: È il DTO che T23 riceve via API quando un client (T5) richiede la creazione di un nuovo messaggio
- *MessageController.java*: espone le API REST di T23 per creare, leggere (inbox/outbox) e cancellare i messaggi, delegando la logica a *MessageService*.
- *MessageService.java*: implementa la logica applicativa della messaggistica, ovvero crea i messaggi, costruisce le inbox/outbox come *MessageDTO* con e-mail utente, verifica autorizzazioni e cancella i messaggi
- *MessageRepository.java*: fornisce i metodi JPA per leggere e scrivere i messaggi nel database
- *NewMessageDTO.java*: È il DTO che T23 restituisce ai client per le inbox/outbox, arricchito con e-mail del mittente/destinatario oltre ai dati base del messaggio

PERSISTENZA DEI MESSAGGI

La gestione della persistenza è affidata a **MessageRepository**, un'interfaccia che estende le funzionalità di Spring Data JPA. Questo componente funge da ponte tra l'applicazione e il database, orchestrando le operazioni sulla tabella dei messaggi basandosi sull'entità *Message*.

Il punto di forza di questa implementazione è la *Query Derivation*. Spring analizza la firma dei metodi definiti nel repository e genera automaticamente il codice SQL necessario seguendo le convenzioni di denominazione.

Ad esempio, il metodo: *List<Message> findByReceiverIdOrderByTimestampDesc(Long receiverId)*;

Produce interamente la seguente istruzione SQL:

SELECT * FROM messages

WHERE receiver_id = ?

ORDER BY timestamp DESC;

Nello specifico, la tabella oggetto di tale query è quella indicata nell'istruzione

@Table (name="messages")

nella classe *Message.java* di T23 e ciò si evince dal fatto che il repository è tipizzato con l'entity *Message*.

L'entità che rappresenta i messaggi presenta il seguente schema relazionale:

MESSAGGI (id, senderId, receiverId, content, timestamp)

Ecco una tabella di esempio:

<u>id</u>	senderId	receiverId	content	timestamp
36	57	2	"Ciao Silvia, sono Ciro"	2026-02-07T15:38:05.980114
24	2	54	"Ciao Pietro, come stai?"	2025-12-11T09:27:37.53383
23	2	54	"ciao Pietro"	2025-12-11T09:19:04.110035

Seguono, in ordine, i rispettivi messaggi:

To: silviapedata03@gmail.com

Ciao Silvia, sono Ciro

07/02/2026, 15:38:05

Delete

To: pietro123@gmail.com

ciao pietro, come stai?

11/12/2025, 09:27:37

Delete

To: pietro123@gmail.com

ciao pietro

11/12/2025, 09:19:04

Delete

È importante notare che i messaggi nel database memorizzano solo gli id di mittente e destinatario (senderId, receiverId) e non le rispettive e-mail. Quindi nel MessageService si usano questi id per recuperare le rispettive e-mail dai profili utente e arricchire il DTO prima di mandarlo al frontend.

Per esempio: in *getInbox*, per ogni Message trovato con *findByReceiverIdOrderByTimestampDesc*, si crea un MessageDTO, copiando i campi base (id, senderId, receiverId, content, timestamp), poi con *msg.getSenderId()* si cerca il UserProfile del mittente tramite *userRepository.findByPlayer_ID(...)* e, se tale UserProfile esiste, si prende *getEmail()* e si effettua *dto.setSenderEmail(...)*, così il client riceve direttamente l'e-mail del mittente senza dover fare altre chiamate.

Funziona allo stesso modo per *getOutbox*.

```
if (msg.getSenderId() != null) {
    UserProfile senderProfile = userRepository.findByPlayer_ID(msg.getSenderId());
    if (senderProfile != null) {
        dto.setSenderEmail(senderProfile.getEmail());
    }
}
```

3. PROGETTAZIONE

In fase di progettazione, è stata valutata l'opportunità di accorpare le funzionalità in un unico servizio (T9). Tuttavia, si è deciso di mantenere la separazione tra i microservizi **T5** e **T23** per preservare il principio di singola responsabilità e garantire una scalabilità indipendente dei moduli.

La strategia adottata prevede una netta distinzione tra lo strato di presentazione e lo strato di persistenza dei dati.

Per garantire il corretto flusso informativo tra i componenti, è stata implementata una stretta collaborazione tramite interfacce di programmazione.

Il modulo T5 (Front-End) interroga il servizio T23 tramite chiamate **API REST** per recuperare in tempo reale il profilo completo.

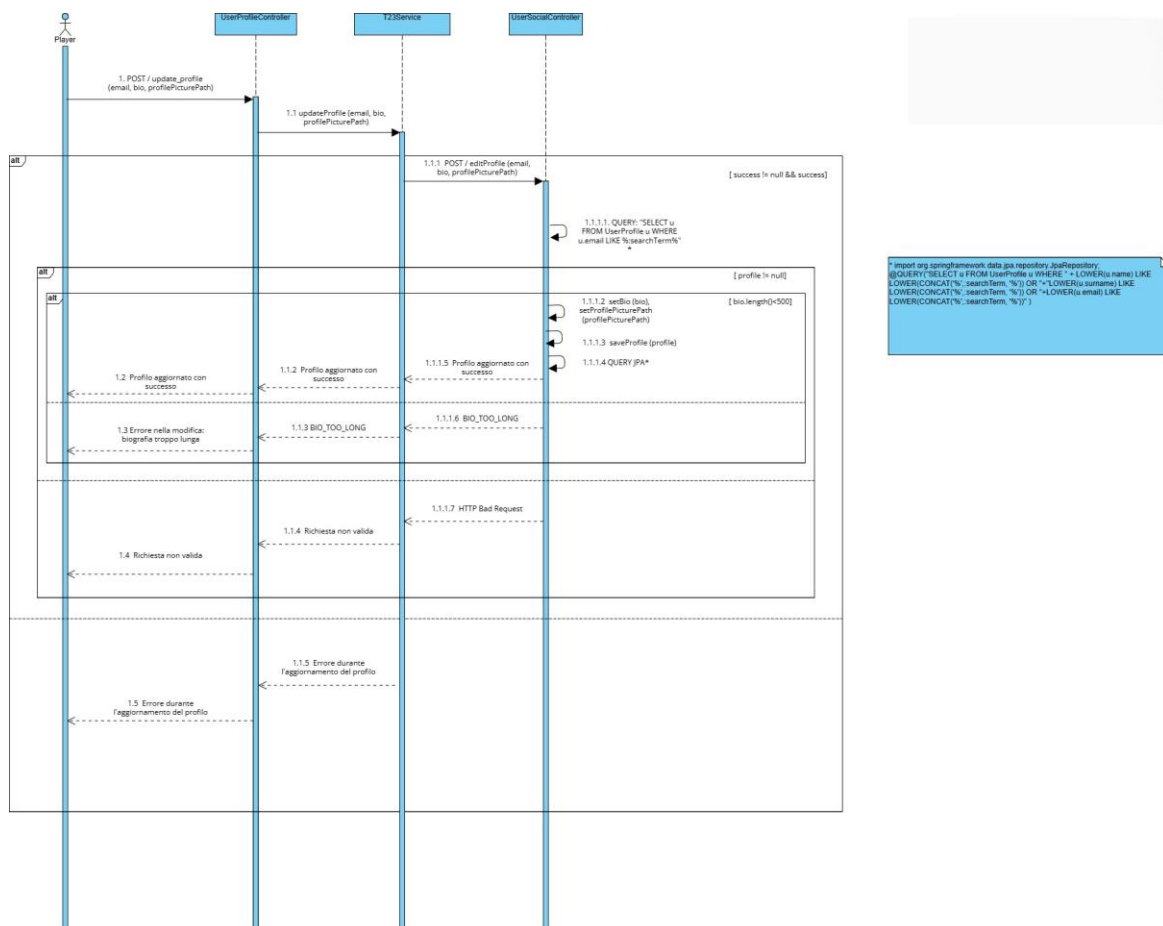
Sono state progettate e implementate nuove API all'interno del microservizio T5 per esporre i dati necessari, garantendo che ogni richiesta dal Front-End riceva una risposta strutturata e coerente con i nuovi modelli di dati.

DIAGRAMMA DI SEQUENZA

I diagrammi di sequenza rappresentano il flusso di interazioni tra gli attori e i componenti del sistema.

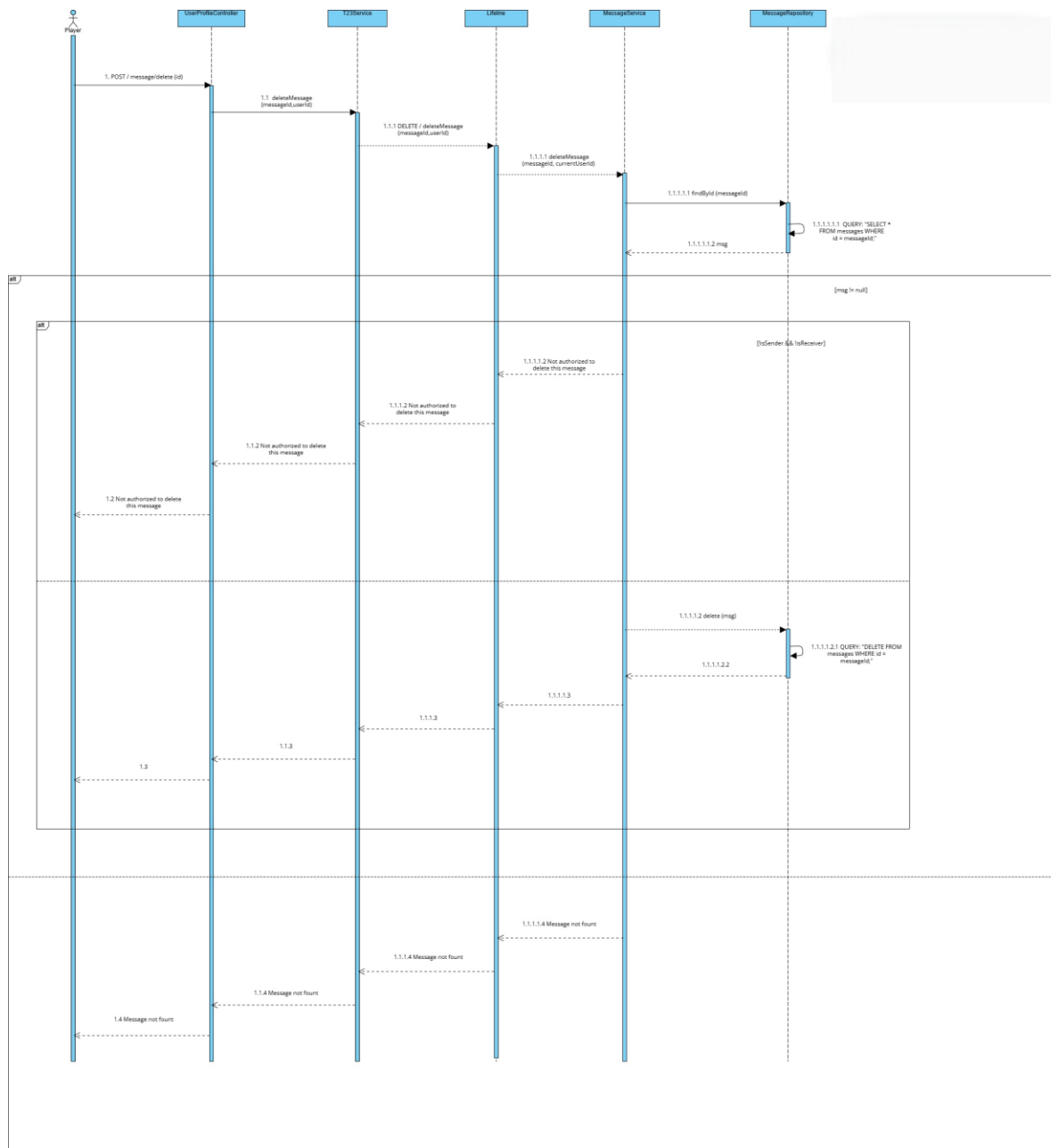
Caso d'uso: ModificaBio

Si mostra come l'utente interagisca con l'interfaccia web per aggiornare la propria biografia o immagine del profilo, come la richiesta venga inviata al microservizio T5 (che gestisce la comunicazione con il frontend) e successivamente inoltrata al microservizio T23, responsabile della gestione dei dati nel database. Il diagramma evidenzia il percorso dei dati, il controllo della validità della bio (ad esempio il limite di 500 caratteri) e la risposta del sistema che, a seconda dell'esito, comunica all'utente il successo dell'operazione o eventuali errori. In questo modo, il diagramma consente di comprendere chiaramente la sequenza di eventi necessaria per garantire l'integrità dei dati e l'interazione corretta tra i microservizi, senza entrare nei dettagli di implementazione.



All'interno della classe `UserSocialController`, è stata implementata una query JPA personalizzata, i cui dettagli tecnici sono riportati nella nota a margine del relativo diagramma di sequenza. Il sistema esegue una ricerca testuale basata sul parametro `:searchTerm`, confrontandolo simultaneamente con i campi `name`, `surname` ed `e-mail` dell'entità `UserProfile`.

Caso d'uso: EliminaMessaggio



All'interno della classe `MessageController` è stato progettato un endpoint REST dedicato alla cancellazione dei messaggi, che espone un'operazione `DELETE` parametrizzata con l'identificativo del messaggio e l'id dell'utente che richiede l'azione. La richiesta viene inoltrata al corrispondente

metodo del `MessageService`, il quale recupera dal database l'entità `Message` attraverso il `MessageRepository` e ne valida l'accesso confrontando l'utente corrente con i campi `senderId` e `receiverId` associati alla conversazione. Solo qualora l'utente risulti effettivamente mittente o destinatario del messaggio, il sistema procede con la rimozione fisica della riga dalla tabella `messages`, propagando al client un esito positivo (204 *No Content*) a conferma dell'avvenuta eliminazione. In tutti gli altri casi – messaggio inesistente o utente non autorizzato – viene sollevata un'eccezione applicativa che causa il rollback della transazione e si traduce in una risposta di errore coerente (ad esempio "Message not found" o "Not authorized to delete this message"), preservando l'integrità e la riservatezza dei dati scambiati tra gli utenti.

REST API

Sono state introdotte nuove REST API all'interno del microservizio T5:

Metodo	Endpoint	Descrizione
POST	/messages/new	Consente la creazione e l'invio di un nuovo messaggio. Riceve dal browser i dati necessari a creare un <i>NewMessageDTO</i> contenente gli identificativi di mittente, destinatario e il corpo del testo. Chiama poi il microservizio T23 per creare effettivamente il messaggio nel database, restituendo una semplice risposta 200/500 alla UI.
GET	/messages/inbox	Recupera la lista di tutti i messaggi ricevuti dall'utente specificato tramite l'ID. Chiama il microservizio T23 per ottenere tutti i messaggi ricevuti e restituisce al browser la lista (eventualmente vuota) come JSON, così la UI può mostrare la posta in arrivo dell'utente.
GET	/messages/outbox	Recupera la cronologia dei messaggi inviati dall'utente loggato, ricavandone l'id. Chiede al microservizio T23 tutti i messaggi inviati da quell'utente e restituisce al browser la lista (o una lista vuota) come JSON, così la UI può mostrare la posta inviata.
DELETE	/messages/delete	Viene chiamata dal bottone "Delete" della UI: riceve l'id del messaggio dal form HTML e recupera l'id dell'utente loggato per delegare al microservizio T23 la cancellazione vera e propria. Qui si usa POST (e non DELETE) perché i form HTML supportano nativamente solo GET e POST.
POST	/update_profile	Gestisce l'aggiornamento della biografia e dell'immagine del profilo. Utilizza il

		<i>serviceManager</i> per inoltrare la richiesta di persistenza al microservizio T23, garantendo l'integrità dei dati dello studente.
GET	<code>/friend/{playerID}</code>	Costruisce la pagina del profilo di un altro giocatore: prende dal path l'id dell'amico, recupera l'id dell'utente loggato dal token, se i due coincidono reindirizza a <code>/profile</code> , altrimenti inizializza un <i>UserProfileComponent</i> in modalità "friend" e, tramite T23, i del giocatore per mostrarli nella sua pagina del profilo.

Ed altre nel microservizio T23:

Metodo	Endpoint	Descrizione
POST	<code>/toggle_follow</code>	Gestisce dinamicamente, con un solo endpoint, la relazione di "Follow" tra due utenti. Viene richiamata dalla UI con il pulsante "Follow/Unfollow" passandovi l'id del profilo target; il sistema inverte lo stato esistente (aggiunge se non presente, rimuove se presente) e restituisce il nuovo stato per aggiornare il testo del bottone.
POST	<code>/messages/new</code>	Espone l'API REST che crea un nuovo messaggio: riceve nel body un <i>NewMessageDTO</i> con <i>senderId</i> , <i>receiverId</i> e <i>content</i> , fa una validazione minima e, se qualcosa manca, risponde con 400 (Bad Request); se i dati sono validi, delega a <i>messageService.createMessage(...)</i> la creazione dell'entità <i>Message</i> e il salvataggio nel database, restituendo infine 200 OK senza corpo
GET	<code>/messages/inbox/{userId}</code>	Restituisce la posta in arrivo di un utente: prende l' <i>userId</i> dal path, chiama <i>messageService.getInbox(userId)</i> per recuperare dal database tutti i messaggi in cui quell'utente è destinatario, li converte in una lista di <i>MessageDTO</i> e li ritorna al client come JSON con 200 OK
GET	<code>/messages/outbox/{userId}</code>	Restituisce la posta inviata di un utente: prende l' <i>userId</i> dal path, chiama <i>messageService.getOutbox(userId)</i> per

		recuperare dal database tutti i messaggi in cui quell'utente è mittente, li converte in una lista di MessageDTO e li ritorna al client come JSON con stato 200 OK
DELETE	/messages/{id}	Riceve l'id del messaggio e l'userId , invoca messageService.deleteMessage(id, userId) che verifica che il messaggio appartenga a quell'utente e lo rimuove dal database, quindi restituisce una risposta senza corpo.
POST	/profile/update_profile	Riceve i parametri (e-mail, bio, profilePicturePath) e aggiorna l'entità UserProfile nel database. Restituisce un booleano per confermare l'esito dell'operazione di salvataggio.
GET	/followers	Chiama userSocialService.getFollowers(userId) e restituisce la lista dei follower dell'utente indicato.
GET	/following	Chiama userSocialService.getFollowing(userId) e restituisce la lista degli utenti che quel userId sta seguendo

Si nota che ci sono due “strati” diversi che espongono URL simili, ma con ruoli diversi:

- In T23 – MessageController
È un REST controller backend puro.
 - Espone l'API “di dominio” dei messaggi:
 - /messages/new: crea il messaggio.
 - /messages/inbox/{userId} :restituisce la inbox di *quel* userId.
 - /messages/outbox/{userId}: restituisce la outbox di *quel* userId.
 - DELETE /messages/{id}?userId=... : cancella un messaggio (con controllo user).
 - Qui i parametri (senderId, receiverId, userId) arrivano dal client che chiama T23 (in pratica T5).
- In T5 – UserProfileController

È un controller “frontend” che si occupa di UI e delle chiamate verso i servizi.

- Espone rotte “di frontiera” per il browser e *dietro le quinte* chiama T23:
 - /messages/new: legge il sender dal token (PageBuilder), setta dto.setSenderId(...) e poi chiama t23Service.sendMessage(...) : che a sua volta chiama T23 /messages/new.
 - /messages/inbox: ricava userId dal token, chiama serviceManager.handleRequest("T23", "GetInboxMessages", userId) che finisce su T23 /messages/inbox/{userId}.

- /messages/delete: ricava userId dal token, chiama T23 "DeleteMessage": che mappa su DELETE /messages/{id}?userId=....
- /messages/outbox: ricava userId dal token e chiama T23 "GetOutboxMessages" → che mappa su GET /messages/outbox/{userId}.

4. IMPLEMENTAZIONE

STRUTTURA DEL PROGETTO MODIFICATO

A seguito dell'analisi dei requisiti e della definizione dell'architettura, si è proceduto alla fase di implementazione che ha visto il coinvolgimento diretto dei microservizi **T23** e **T5**. L'attività di sviluppo è stata caratterizzata da un duplice approccio: da un lato l'estensione delle capacità del sistema tramite l'introduzione di nuovi componenti, soprattutto per la funzionalità dei messaggi introdotta dal team, dall'altro l'adeguamento dei moduli preesistenti per supportare le nuove funzionalità social e di gestione del profilo.

Nella tabella sottostante vengono dettagliati i moduli software oggetto dell'intervento, distinguendo tra i componenti creati *ex-novo* e quelli che hanno subito un processo di refactoring o aggiornamento:

Moduli Aggiunti	Moduli Modificati
Message.java (in T5)	profile.html
Message.java (in T23)	profile.js
NewMessageDTO.java (in T23)	Edit_Profile.html
MessageDTO.java	profile-edit.js
MessageController.java	UserProfileController.java
MessageService.java	T23Service.java
MessageRepository.java	UserSocialController.java
NewMessageDTO.java (in T5)	

Spiegazione delle aggiunte

Si è deciso, per facilitare la comunicazione tra utenti e lo scambio di informazioni, di introdurre delle funzionalità per la messaggistica. Sono state introdotte nuove classi apposite ed i moduli **profile.html** e **profile.js** sono stati modificati per permettere la corretta visualizzazione ed interazione con i messaggi. Le modifiche apportate a questi due moduli sono approfondite nel paragrafo "Spiegazione delle modifiche/refactoring".

I messaggi sono gestiti come una piccola messaggistica privata tra utenti, con una catena completa Frontend – T5 – T23 – database che supporta creazione, visualizzazione di inbox/outbox e cancellazione controllata dei messaggi.

```
@Entity
@Table(name = "messages")
public class Message {

    @Id 2 usages
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private Long senderId; 2 usages
    private Long receiverId; 2 usages

    @Column(nullable = false, length = 1000) 2 usages
    private String content;

    @Column(nullable = false) 2 usages
    private LocalDateTime timestamp = LocalDateTime.now();
}
```

Classe Message.java di T23. Sono presenti anche gli appositi metodi getter e setter

Ogni record memorizza il mittente (senderId), il destinatario (receiverId), il testo (content) e la data/ora (timestamp).

Per non esporre direttamente l'entità JPA, il Back-end T23 usa un DTO. Infatti, in T23 la classe Message è un'entità JPA: rappresenta come il messaggio è salvato nel database (tabella, colonne, con annotazioni @Entity, @Column, ecc.).

Il MessageDTO invece è un Data Transfer Object: rappresenta cosa viene scambiato via API (solo i campi necessari verso l'esterno, senza dettagli di persistenza) ed è l'oggetto che viaggia tra microservizio e altri componenti.

```

public class MessageDTO { 2 usages
    private Long id; 2 usages
    private Long senderId; 2 usages
    private Long receiverId; 2 usages
    private String content; 2 usages
    private LocalDateTime timestamp; 2 usages
    private String senderEmail; 2 usages
    private String receiverEmail; 2 usages

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public Long getSenderId() { return senderId; }
    public void setSenderId(Long senderId) { this.senderId = senderId; }

    public Long getReceiverId() { return receiverId; }
    public void setReceiverId(Long receiverId) { this.receiverId = receiverId; }

    public String getContent() { return content; }
    public void setContent(String content) { this.content = content; }

    public LocalDateTime getTimestamp() { return timestamp; }
    public void setTimestamp(LocalDateTime timestamp) { this.timestamp = timestamp; }

    public String getSenderEmail() { return senderEmail; } no usages
    public void setSenderEmail(String senderEmail) { this.senderEmail = senderEmail; } no usages

    public String getReceiverEmail() { return receiverEmail; } no usages
    public void setReceiverEmail(String receiverEmail) { this.receiverEmail = receiverEmail; }
}

```

La inbox è l'insieme dei messaggi in cui l'utente è destinatario; la outbox è l'insieme dei messaggi in cui l'utente è mittente. T23 distingue i due casi usando i metodi del MessageRepository:

```

// inbox di un utente
List<Message> findByReceiverIdOrderByTimestampDesc(Long receiverId);

// messaggi inviati da un utente (outbox)
List<Message> findBySenderIdOrderByTimestampDesc(Long senderId); no

```

Nel servizio applicativo (MessageService) questi risultati vengono trasformati in MessageDTO e arricchiti con l'e-mail di mittente o destinatario tramite UserProfileRepository.

Inbox:

```
@Transactional(readOnly = true)
public List<MessageDTO> getInbox(Long userId) {
    List<Message> raw = messageRepository.findByReceiverIdOrderByTimestampDesc(userId);
    List<MessageDTO> dtos = new ArrayList<>();

    for (Message msg : raw) {
        MessageDTO dto = new MessageDTO();
        dto.setId(msg.getId());
        dto.setSenderId(msg.getSenderId());
        dto.setReceiverId(msg.getReceiverId());
        dto.setContent(msg.getContent());
        dto.setTimestamp(msg.getTimestamp());

        if (msg.getSenderId() != null) {
            UserProfile senderProfile = userProfileRepository.findByPlayer_ID(msg.getSenderId());
            if (senderProfile != null) {
                dto.setSenderEmail(senderProfile.getEmail());
            }
        }

        dtos.add(dto);
    }

    return dtos;
}
```

Outbox:

```
@Transactional(readOnly = true)
public List<MessageDTO> getOutbox(Long userId) {
    List<Message> raw = messageRepository.findBySenderIdOrderByTimestampDesc(userId);
    List<MessageDTO> dtos = new ArrayList<>();

    for (Message msg : raw) {
        MessageDTO dto = new MessageDTO();
        dto.setId(msg.getId());
        dto.setSenderId(msg.getSenderId());
        dto.setReceiverId(msg.getReceiverId());
        dto.setContent(msg.getContent());
        dto.setTimestamp(msg.getTimestamp());

        // Mostra email del DESTINATARIO nell'outbox
        if (msg.getReceiverId() != null) {
            UserProfile receiverProfile = userProfileRepository.findByPlayer_ID(msg.getReceiverId());
            if (receiverProfile != null) {
                dto.setReceiverEmail(receiverProfile.getEmail());
            }
        }

        dtos.add(dto);
    }

    return dtos;
}
```

Quindi:

- Nell'inbox si visualizza soprattutto il mittente (senderEmail);
- Nell'outbox si visualizza soprattutto il destinatario (receiverEmail).

La cancellazione è protetta: solo chi è mittente o destinatario può eliminare un messaggio.

```

@Transactional
public void deleteMessage(Long messageId, Long currentUserId) {
    Message msg = messageRepository.findById(messageId)
        .orElseThrow(() -> new RuntimeException("Message not found"));

    boolean isSender = currentUserId.equals(msg.getSenderId());
    boolean isReceiver = currentUserId.equals(msg.getReceiverId());

    if (!isSender && !isReceiver) {
        // Se l'utente non è né chi ha inviato né chi ha ricevuto, blocchiamo l'azione
        throw new RuntimeException("Not authorized to delete this message");
    }

    messageRepository.delete(msg);
}

```

In T23, il MessageController espone le API REST “pure” per la messaggistica, pensate per essere consumate da altri servizi

```

@RestController
@RequestMapping("/messages")
@CrossOrigin(origins = "http://localhost:8085")
public class MessageController {

    private static final Logger logger = LoggerFactory.getLogger(MessageController.class);

    private final MessageService messageService;

    public MessageController(MessageService messageService) { this.messageService = messageService; }

    @PostMapping("/new")
    public ResponseEntity<Void> createMessage(@RequestBody NewMessageDTO dto) {...}

    @GetMapping("/inbox/{userId}")
    public ResponseEntity<List<MessageDTO>> inbox(@PathVariable Long userId) {...}

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteMessage(
        @PathVariable Long id,
        @RequestParam Long userId) {...}

    @GetMapping("/outbox/{userId}")
    public ResponseEntity<List<MessageDTO>> outbox(@PathVariable Long userId) {...}
}

```

Qui (con l’endpoint /message/new) si usa NewMessageDTO lato T23:


```
public class NewMessageDTO {  
  
    private Long senderId;  
    private Long receiverId;  
    private String content;  
    private String senderEmail;  
  
    public Long getSenderId() { return senderId; }  
    public void setSenderId(Long senderId) { this.senderId = senderId; }  
  
    public Long getReceiverId() { return receiverId; }  
    public void setReceiverId(Long receiverId) { this.receiverId = receiverId; }  
  
    public String getContent() { return content; }  
    public void setContent(String content) { this.content = content; }  
  
    public String getSenderEmail() { return senderEmail; }  
    public void setSenderEmail(String email) { this.senderEmail = senderEmail; }  
}
```

Nel front-end T5, invece, non si parla direttamente con questo controller: si passa per UserProfileController di T5, che fa da bridge verso T23 tramite ServiceManager/T23Service.

Per i messaggi T5 definisce un proprio Message e NewMessageDTO (con la stessa struttura ma senza annotazioni JPA) usati verso il browser.

```

public class Message {
    private Long id; 3 usages
    private Long senderId; 3 usages
    private Long receiverId; 3 usages
    private String content; 3 usages
    private LocalDateTime timestamp; 3 usages
    private String senderEmail; 2 usages
    private String receiverEmail; 2 usages

    public Message() {
    }

    public Message(Long id, Long senderId, Long receiverId, String content, LocalDateTime timestamp) {
        this.id = id;
        this.senderId = senderId;
        this.receiverId = receiverId;
        this.content = content;
        this.timestamp = timestamp;
    }

    public Long getId() { return id; }

    public void setId(Long id) { this.id = id; }

    public Long getSenderId() { return senderId; }

    public void setSenderId(Long senderId) { this.senderId = senderId; }

    public Long getReceiverId() { return receiverId; }
}

```

Estratto di Message.java di T5

```

public class NewMessageDTO {

    private Long senderId;
    private Long receiverId;
    private String content;
    private String senderEmail;

    public Long getSenderId() { return senderId; }
    public void setSenderId(Long senderId) { this.senderId = senderId; }

    public Long getReceiverId() { return receiverId; }
    public void setReceiverId(Long receiverId) { this.receiverId = receiverId; }

    public String getContent() { return content; }
    public void setContent(String content) { this.content = content; }

    public String getSenderEmail() { return senderEmail; }
    public void setSenderEmail(String email) { this.senderEmail = email; }

}

```

Il UserProfileController di T5 espone dunque endpoint specifici che usano questi tipi e delegano a T23. Seguono due endpoint di esempio:

Outbox:

```

@GetMapping("/messages/outbox") no usages
public ResponseEntity<List<Message>> getOutboxMessages() {
    try {
        PageBuilder page = new PageBuilder(serviceManager, "profile", null, JwtRequestContext.getJwtToken());
        Long userId = page.getUserId();

        @SuppressWarnings("unchecked")
        List<Message> messages = (List<Message>) serviceManager
            .handleRequest("T23", "GetOutboxMessages", userId);

        return ResponseEntity.ok(messages != null ? messages : new ArrayList<>());
    } catch (Exception e) {
        logger.error("Errore getOutboxMessages", e);
        return ResponseEntity.ok(new ArrayList<>());
    }
}

```

Sono presenti gli endpoint analoghi /message/new, /message/inbox.

In T5 l'azione di cancellazione è legata a un pulsante nella pagina: quando l'utente clicca "Delete", il browser invia la richiesta tramite un form HTML. Poiché i form HTML supportano solo GET e POST, non si può usare, in questo specifico contesto, direttamente DELETE, e ho esposto un endpoint POST /messages/delete che internamente richiama il DELETE /messages/{id} di T23.

```

@PostMapping("/messages/delete") no usages
public ResponseEntity<Void> deleteInboxMessagePost(@RequestParam Long id) {
    PageBuilder page = new PageBuilder(serviceManager, "profile", null, JwtRequestContext.getJwtToken());
    Long userId = page.getUserId(); // ricavato dal token/sessione

    logger.info("POST /messages/delete id={} userId={}", id, userId);
    serviceManager.handleRequest("T23", "DeleteMessage", id, userId);
    return ResponseEntity.noContent().build();
}

```

Quindi si usa un POST semantico "delete" che internamente chiama comunque il servizio T23 DeleteMessage (che a sua volta usa MessageService.deleteMessage), mantenendo la logica di autorizzazione e la DELETE a livello di dominio, ma evitando i problemi tecnici della DELETE HTTP sul livello front-end. Di fatto a livello logico rimane una delete, ma a livello HTTP si usa un POST per bypassare le difficoltà tecniche del DELETE in questo contesto.

Inoltre, è importante notare che per la messaggistica si è scelto di mantenere distinti i modelli lato front-end (T5) e lato microservizio di dominio (T23). In T23 la classe Message rappresenta l'entità JPA mappata sul database, mentre MessageDTO e NewMessageDTO sono i DTO esposti dalle API del servizio. In T5 esistono classi omonime Message, NewMessageDTO che hanno struttura analoga, ma sono dedicate alla serializzazione verso il browser e non contengono annotazioni JPA né logica di persistenza. Questa duplicazione controllata permette di disaccoppiare il front-end dal modello interno del microservizio: T23 può evolvere lo schema o arricchire i DTO senza impattare direttamente il codice JavaScript, e T5 può aggiungere campi specifici per la UI. In effetti, in ottica microservizi è preferibile accettare una minima ripetizione di struttura rispetto a un modello condiviso rigido, perché riduciamo il coupling tra servizi e manteniamo la possibilità di deploy indipendenti.

Spiegazione delle modifiche/refactoring

In fase di sviluppo, è stato condotto un significativo intervento di refactoring sulla pagina del profilo, finalizzato a una più netta separazione delle responsabilità tra la struttura del documento (HTML) e la logica applicativa (JavaScript). Nello specifico, l'attività si è concentrata sulla riorganizzazione dei file *profile.html* e *profile.js*.

Inizialmente, la gestione delle interfacce relative alle sezioni social, messaggistica e interazioni tra utenti (follow/unfollow) risultava integrata direttamente all'interno di *profile.html*. Tale architettura, caratterizzata da script eccessivamente estesi nel corpo della pagina, comprometteva la leggibilità del codice e la riusabilità delle funzioni.

Il nuovo approccio implementato prevede che:

- *profile.html* assolva esclusivamente al ruolo di definizione del markup e dei placeholder necessari alla visualizzazione.
- *profile.js* centralizzi l'intera logica client-side, venendo richiamato dal file HTML tramite un tag `<script>` dedicato.

Questa suddivisione strutturale apporta benefici immediati in termini di manutenibilità ed estensibilità del sistema, agevolando la futura integrazione di moduli aggiuntivi per notifiche, team e messaggistica. Inoltre, la modularizzazione consente il riutilizzo della logica JavaScript in viste correlate, eliminando la ridondanza del codice.

Infine, gli interventi hanno permesso un'evoluzione dell'interfaccia utente, ora organizzata in una vista ibrida:

1. **Componente Statica:** sempre visibile, dedicata alle informazioni persistenti del profilo.
2. **Componente Dinamica:** gestita tramite un sistema a schede (*tab*), che permette di navigare agilmente tra le sezioni *Games*, *Social*, *Teams* e *Notifications*.

L'intera transizione verso file logici dedicati garantisce un codice più chiaro, un'esecuzione più efficiente e una struttura pronta per futuri incrementi funzionali.

Un esempio è il seguente codice che indica come viene popolata parte della colonna di sinistra con alcune delle informazioni di interesse.

```
<!-- Sinistra: card profilo -->
<div class="col-md-4 mb-3">
  <div class="card h-100 my-3 p-0">
    <div class="row g-0">
      <div class="col-12 text-center p-3">
        <!-- Name e Surname -->
        <h3 class="mb-2" th:text="|${user.userProfile.name} ${user.userProfile.surname}|"></h3>

        <!-- Immagine -->
        <div class="d-inline-block mb-2">
          
        </div>

        <!-- Email piccola -->
        <p class="mb-2">
          <small id="user-email" th:text="${user.email}"></small>
        </p>
      </div>
    </div>
  </div>
</div>
```

Sempre nella parte statica sono previsti i contatori di followers e following, inizialmente vuoti e riempiti via JavaScript:

```
<div class="row row-cols-1 row-cols-md-2 g-4 justify-content-center my-3">
  <div class="col text-center">
    <i class="fa fa-check fs-6 d-block mb-2"></i>
    <h4 class="mb-0 fw-semibold lh-1">
      <span id="followersCount"></span>
    </h4>
    <p class="mb-0 fs-5">Followers</p>
  </div>
  <div class="col text-center">
    <i class="fa fa-check fs-6 d-block mb-2"></i>
    <h4 class="mb-0 fw-semibold lh-1">
      <span id="followingCount"></span>
    </h4>
    <p class="mb-0 fs-5">Following</p>
  </div>
</div>
```

Subito sotto le statistiche, *profile.html* definisce la parte dinamica mediante un menu a tab (Games, Social, Teams, Notifications). Il markup dei pulsanti è, ad esempio:

```
<!-- Tab selezione -->
<ul
  class="nav justify-content-end mt-2"
  id="pills-tab"
  role="tablist"
>
  <li class="nav-item" role="presentation">
    <button
      class="nav-link position-relative rounded-0 active d-flex align-items-center justify-content-center bg-transparent fs-5 py-6"
      id="pills-games-tab"
      data-bs-toggle="pill"
      data-bs-target="#pills-games"
      type="button"
      role="tab"
      aria-controls="pills-games"
      aria-selected="true"
    >
      <i class="fa fa-user me-2 fs-4"></i>
      <span class="d-none d-md-block">Games</span>
    </button>
  </li>
```

In questa figura si vede come è stato inserito il pulsante Games nel menù a tab

```

<!-- Bottoni -->
<div class="container my-5">
  <div class="d-flex justify-content-center gap-4 flex-wrap">
    <!-- Achievements -->
    <a href="/Achievement" class="btn btn-warning d-flex flex-column justify-content-center align-items-center px-5 py-4"
      style="text-decoration: none;">
      <i class="bi bi-trophy-fill fs-1 mb-2"></i>
      <span class="fw-bold fs-5">Achievements</span>
    </a>

    <!-- Game History -->
    <a href="/Games" class="btn btn-danger d-flex flex-column justify-content-center align-items-center px-5 py-4"
      style="text-decoration: none;">
      <i class="bi bi-controller fs-1 mb-2"></i>
      <span class="fw-bold fs-5">Game History</span>
    </a>

    <!-- Leaderboard -->
    <a href="/Leaderboard" class="btn btn-primary d-flex flex-column justify-content-center align-items-center px-5 py-4"
      style="text-decoration: none;">
      <i class="bi bi-list-ol fs-1 mb-2"></i>
      <span class="fw-bold fs-5">Leaderboard</span>
    </a>
  </div>
</div>

```

Le tab Teams e Notifications sono state lasciate quasi inalterate: profile.html contiene ancora le card di visualizzazione dei team e la lista delle notifiche, ma la logica JavaScript non è stata estesa perché mancavano i servizi completi lato backend, che sono a cura di altri Team di sviluppo.

Per followers/following è stata creata una funzione riutilizzabile: si tratta della funzione *fetchAndUpdateUserBlock* e si trova nel file profile.js, nella sezione che gestisce la parte Social della pagina profilo (followers/following).

Sempre in profile.js sono stati gestiti i messaggi: la tab Messages chiama loadMessages (mode), che carica via fetch /messages/inbox o /messages/outbox, costruisce dinamicamente le card dei messaggi e permette la cancellazione.

Inoltre, profile.js gestisce il follow/unfollow sulla pagina amico con una logica dedicata: all'apertura interroga /isFollowing?followerId=...&followingId=... e aggiorna il testo del bottone; al click chiama /toggle_follow e sincronizza il nuovo stato sia nella UI sia nel localStorage.

Quando un utente, tramite interfaccia, apre "New chat", sceglie se pescare da followers/following, seleziona utente e scrive il contenuto. Avvengono i seguenti step:

- *profile.js* invia un POST /messages/new (T5) con { senderId, receiverId, content }.
- *UserProfileController* ricava il vero senderId dal token, chiama t23Service.sendMessage, che in T23 costruisce un NewMessageDTO e chiama MessageService.createMessage.
- *MessageService* salva una nuova entità Message in DB

Per le specifiche legate a biografia, immagine profilo e gestione errori, è stata creata una pagina di editing dedicata (edit-profile.html) e uno script separato (profile-edit.js) invece di usare codice inline.

In *Edit_profile.html* la struttura è minimale: un text area per la biografia, una griglia di immagini predefinite e due alert nascosti per mostrare l'esito. Ne presentiamo un estratto:

```

<!-- Selezione immagine profilo -->
<div id="profile-pictures">
  
</div>

<!-- Messaggi di esito -->
<div class="alert alert-success d-none" id="profileSuccess">
  Profilo aggiornato con successo.
</div>
<div class="alert alert-danger d-none" id="profileError">
  Errore nell'aggiornamento del profilo: biografia troppo lunga.
</div>

<!-- Pulsante per salvare -->
<button type="button" id="save-button" th:text="#{editProfile.save}">Salva</button>
</main>

```

Nel corrispondente profile-edit.js la logica è concentrata sulla selezione dell'immagine e sull'invio della richiesta al backend (/api/gameEngine/update_profile) con gestione elegante degli errori.

Questo refactoring ha permesso di centralizzare la logica di aggiornamento del profilo, avere messaggi di errore più chiari e riutilizzare le stesse immagini predefinite anche nella visualizzazione del profilo principale.

```

@PostMapping("/update_profile")
public ResponseEntity<String> updateProfile(
    @RequestParam String email,
    @RequestParam String bio,
    @RequestParam String profilePicturePath) {
    try {
        // Chiama direttamente il servizio T23 per aggiornare il profilo
        Boolean success = (Boolean) serviceManager.handleRequest(
            "T23",
            "UpdateProfile",
            email,
            bio,
            profilePicturePath
        );

        if (success != null && success) {
            return ResponseEntity.ok("Profilo aggiornato con successo");
        } else {
            return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
                .body("Errore durante l'aggiornamento del profilo");
        }
    } catch (IllegalArgumentException e) {
        if ("BIO_TOO_LONG".equals(e.getMessage())) {
            return ResponseEntity.status(HttpStatus.BAD_REQUEST)
                .body("BIO_TOO_LONG");
        }
        // altro caso di IllegalArgumentException
        return ResponseEntity.status(HttpStatus.BAD_REQUEST)
            .body("Richiesta non valida");
    } catch (Exception e) {
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body("Errore durante l'aggiornamento del profilo: " + e.getMessage());
    }
}

```

Durante l'implementazione della funzionalità di aggiornamento del profilo, il flusso inizia nel microservizio T5 con il metodo `UserProfileController.updateProfile`, che raccoglie i dati inviati dal frontend (email, bio, profilePicturePath) e li inoltra al microservizio T23 tramite il `serviceManager`.


```
// Metodo per modificare il profilo di un utente
private Boolean updateProfile(String userEmail, String bio, String imagePath) { 1 usage
    final String endpoint = "/profile/update_profile";
    MultiValueMap<String, String> map = new LinkedMultiValueMap<>();
    map.add(EMAIL_FIELD, userEmail);
    map.add("bio", bio);
    map.add("profilePicturePath", imagePath);
    map.add("nickname", ""); // <-- aggiunto così non fa 400
    return callRestPost(endpoint, map, null, Boolean.class);
}
```

Nel servizio di T5, il metodo T23Service.updateProfile costruisce la richiesta REST verso T23 utilizzando MultiValueMap e invia i dati all'endpoint /profile/update_profile.

```
@PostMapping("/profile/update_profile")
public ResponseEntity<Boolean> editProfile(@RequestParam("email") String email,
                                           @RequestParam("bio") String bio,
                                           @RequestParam("profilePicturePath") String profilePicturePath,
                                           @RequestParam("nickname") String nickname) {

    UserProfile profile = playerService.findProfileByEmail(email);
    if (profile == null) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(false); // Ritorna false in caso di errore
    }

    //if (bio != null && bio.length() > 500) {
    if (bio.length() > 500) {

        throw new IllegalArgumentException("BIO_TOO_LONG");
    }

    profile.setBio(bio);
    profile.setProfilePicturePath(profilePicturePath);
    profile.setNickname(nickname);
    playerService.saveProfile(profile);
    return ResponseEntity.ok(true); // Ritorna true se l'operazione ha avuto successo
}
```

Sul microservizio T23, il metodo UserSocialController.editProfile riceve i parametri, recupera il profilo corrispondente dal database e verifica la validità della bio. Se la bio è valida, aggiorna i campi del profilo e salva l'entità. Il metodo ritorna true in caso di successo. In caso di eccezioni, come bio troppo lunga, T5 intercetta l'errore nel blocco catch (IllegalArgumentException e) e trasmette al frontend un messaggio specifico (BIO_TOO_LONG). Il file profile-edit.js gestisce questa risposta, mostrando all'utente l'alert appropriato o un messaggio generico in caso di altri problemi.

In questo modo il flusso implementativo copre tutto il percorso dei dati: dal frontend, attraverso T5 e T23, fino all'aggiornamento sicuro del database e alla gestione degli errori.

5. TESTING

Il test end-to-end è stato eseguito direttamente tramite l'interfaccia web dell'applicazione, validando che i principali casi d'uso funzionino davvero da schermo a schermo, come in uno scenario reale.

ID	Descrizione e Pre-condizioni	Input	Output (Atteso / Ottenuto)	Post-Condizioni Attese	Esito
1	Invio Messaggio Il player si trova nella sezione social, ha cliccato su "New Chat" e selezionato il destinatario.	Messaggio: "Ciao"	Atteso: Messaggio inviato. Ottenuto: Messaggio Inviato.	La pagina mostra un avviso che conferma il corretto invio del messaggio.	PASS
2	Invio Messaggio (Overload) Il player si trova nella sezione social, ha cliccato su "New Chat" e selezionato il destinatario.	Messaggio: "aaaa...aa" (più di 1000 caratteri)	Atteso: Errore: messaggio troppo lungo. Ottenuto: Errore: messaggio troppo lungo.	La pagina mostra un avviso di errore.	PASS
3	Eliminazione Messaggio Il player decide di eliminare un messaggio nella sezione social.	Click sul pulsante delete.	Atteso: - Ottenuto: -	Il messaggio viene eliminato.	PASS
4	Ricerca Utenti Il player si trova nella sezione Social e clicca il pulsante Search.	Inserimento caratteri e-mail utente cercato.	Atteso: - Ottenuto: -	Appare l'utente corretto, la cui e-mail contiene i caratteri specificati.	PASS
5	Richiesta Follow Il player in sezione social cerca un utente tramite nome e clicca Follow.	Click sul pulsante Follow.	Atteso: Il pulsante diventa Unfollow. Ottenuto: Il pulsante diventa Unfollow.	L'utente appare nei followed; Following player +1; Followers target +1.	PASS

6	Visualizzazione Profilo Terzo Il player si trova nella sezione social e clicca il pulsante View.	Click sul pulsante View.	Atteso: Apertura pagina (parte statica). Ottenuto: Apertura pagina (parte statica).	Si apre la pagina dell'utente selezionato, dove si visualizza solo la parte statica.	PASS
7	Modifica Profilo (Validazione) Il player in sezione social clicca edit profile.	Bio: "aaaa...a" (500 caratteri)	Atteso: Errore: biografia troppo lunga. Ottenuto: Errore: biografia troppo lunga.	Il profilo non risulta modificato. Il player rimane sulla schermata di modifica.	PASS
8	Modifica Profilo (Successo) Il player in sezione social clicca edit profile.	Bio: "Giona"	Atteso: Profilo aggiornato. Ottenuto: Profilo aggiornato.	Il profilo risulta modificato. Il player viene reindirizzato alla pagina del profilo.	PASS

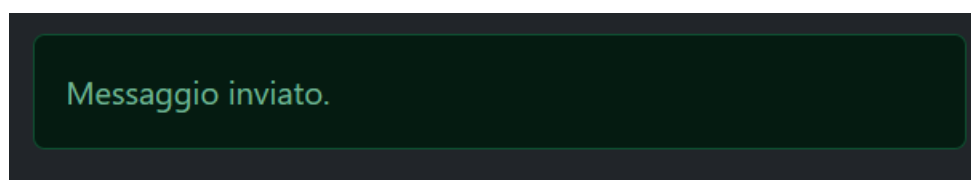


Figura 1: relativa al Test ID 1

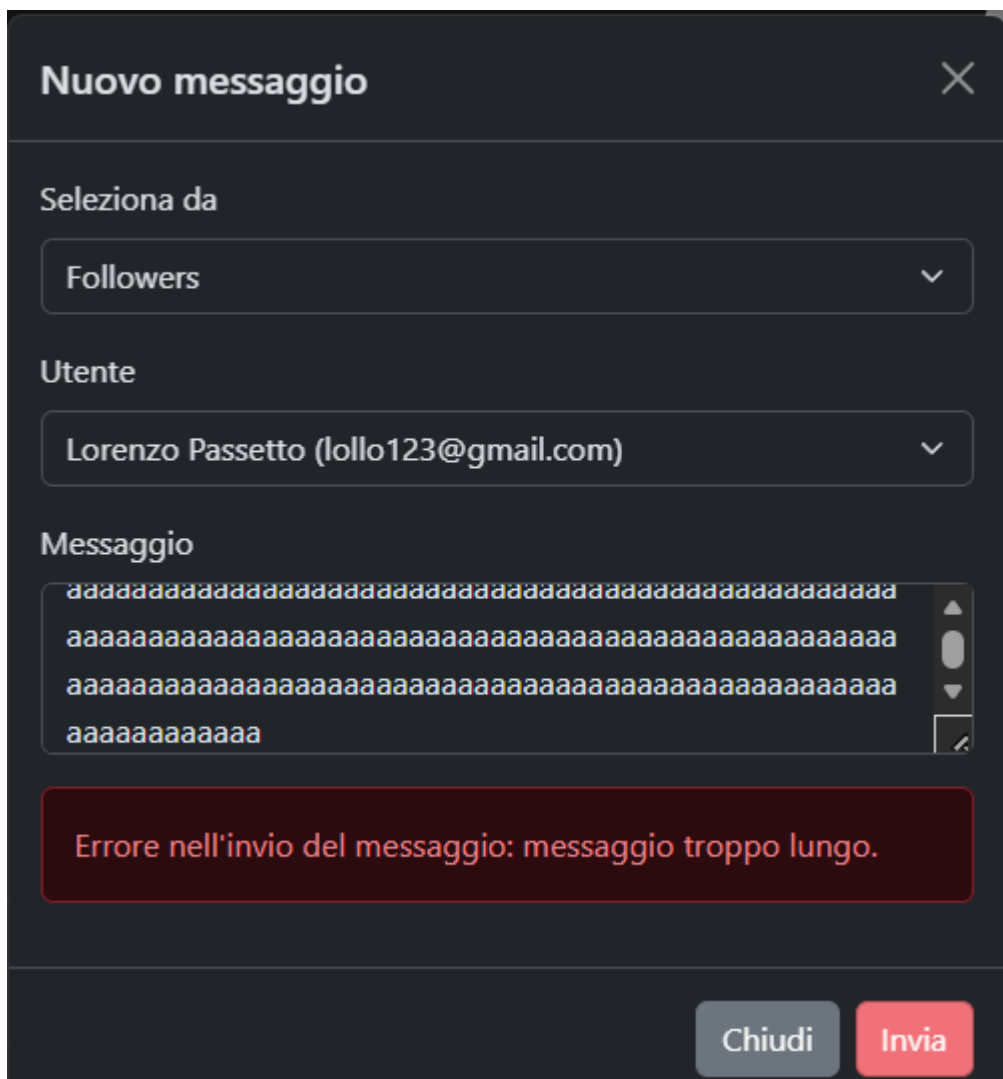


Figura2: relativa al Test ID 2

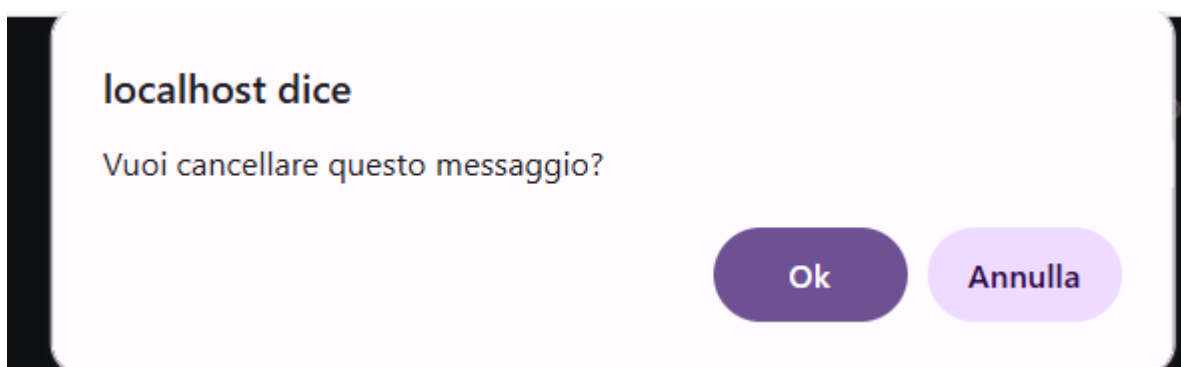


Figura 3: relativa al Test ID 3

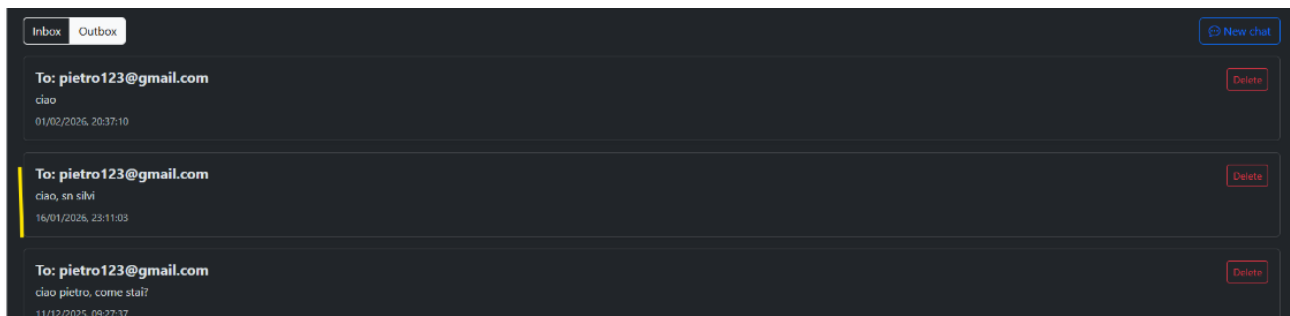


Figura 4: relativa al Test ID 3

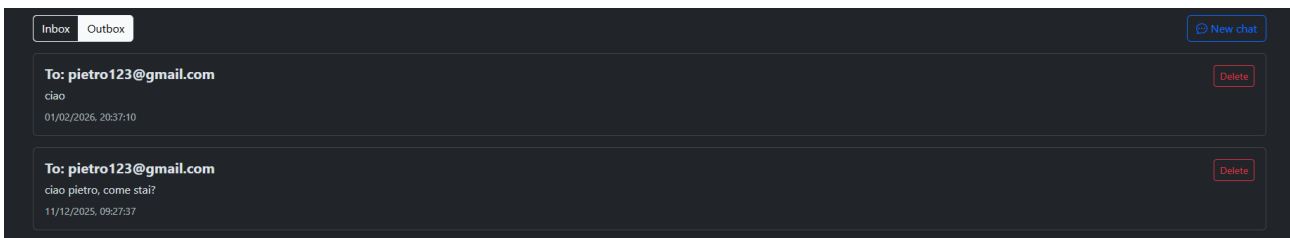


Figura 5: relativa al Test ID 3

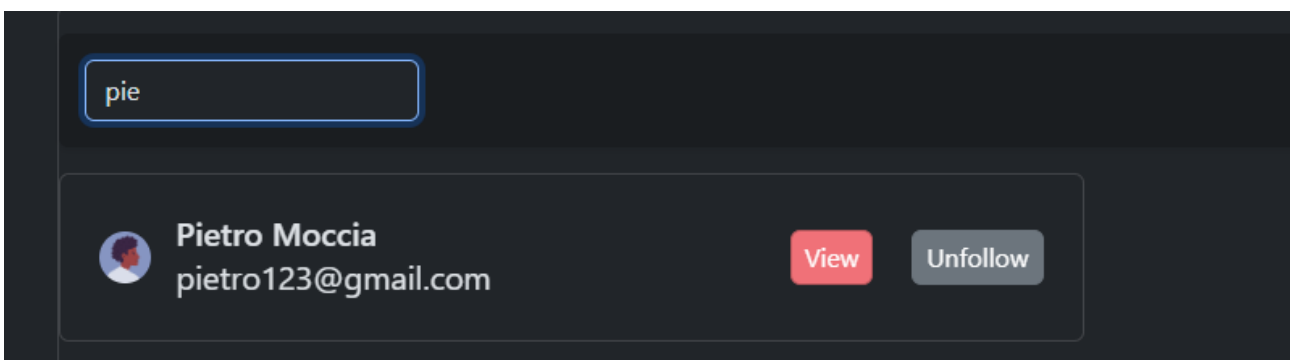


Figura 6: relativa al Test ID 4

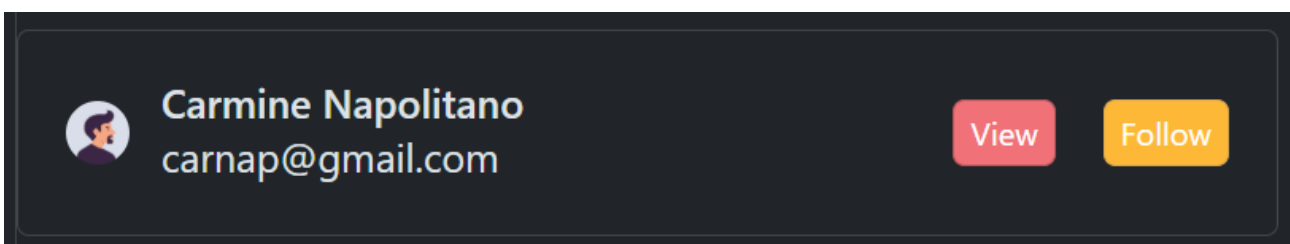


Figura 7: relativa al Test ID 5

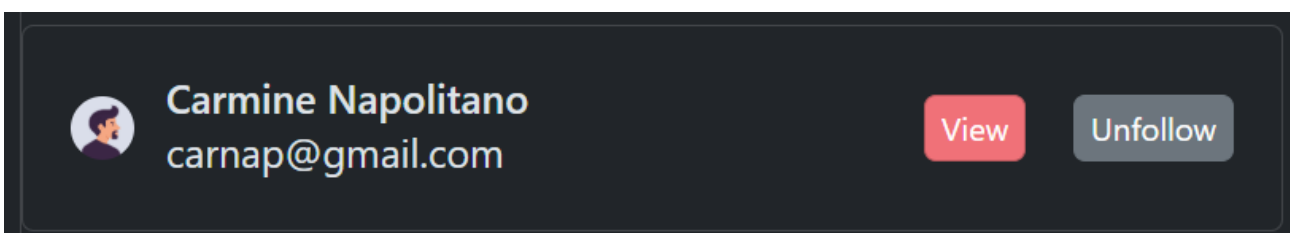


Figura 8: relativa al Test ID 5



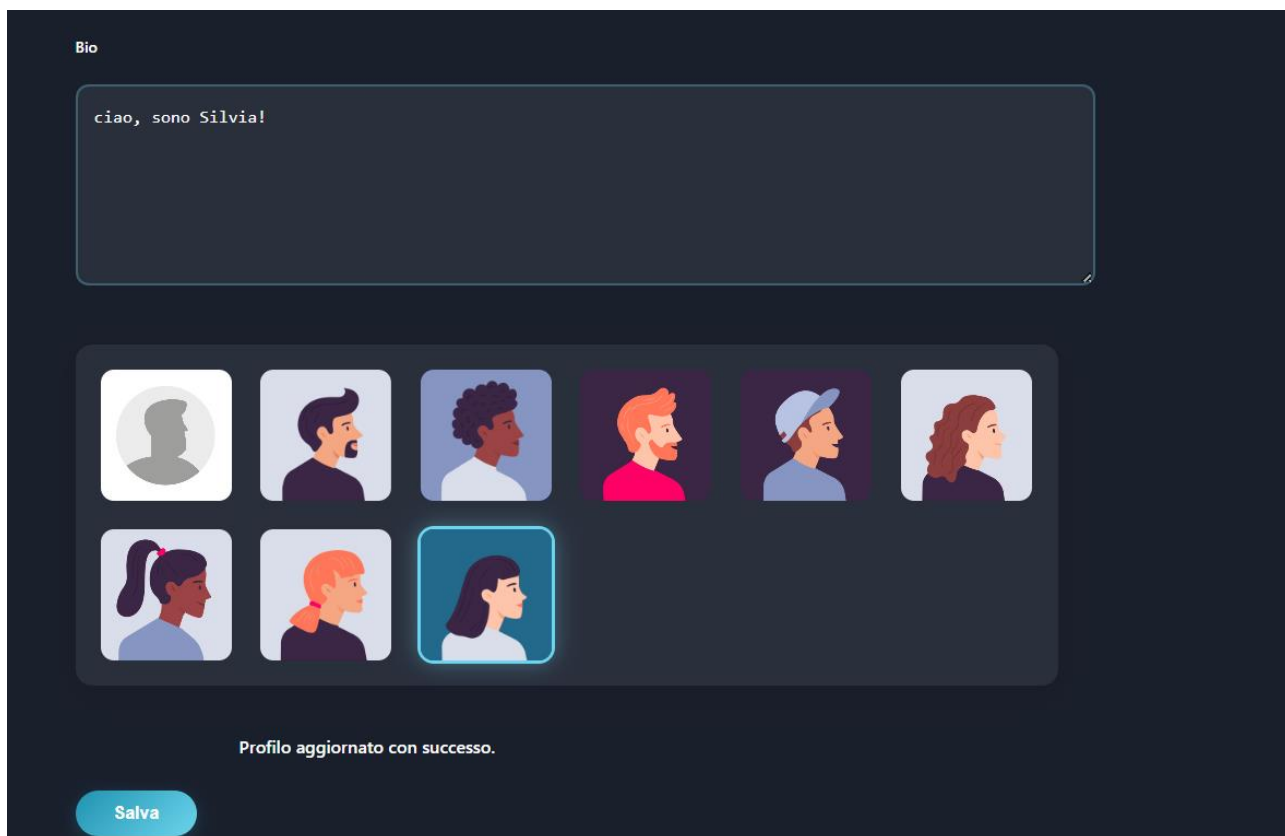


Figura 11: relativa al Test ID 8

6. SVILUPPI FUTURI E SUGGERIMENTI

In futuro si prevede l'integrazione del sistema di notifiche, sviluppata da un altro team di sviluppo, così da informare in tempo reale gli utenti su nuovi messaggi, richieste di amicizia, inviti ai team e altro. Questo permetterebbe una fruizione più dinamica del portale e ridurrebbe il rischio che comunicazioni importanti passino inosservate.

Un ulteriore sviluppo riguarda la gestione dei team: sarebbe opportuno abilitare gli amministratori alla creazione, modifica e cancellazione dei team, permettendo loro di impostare regole di accesso, definire ruoli interni e gestire le immagini associate al gruppo. In questo modo si faciliterebbe la formazione di comunità strutturate e la partecipazione organizzata alle attività di gioco.

Dal punto di vista della personalizzazione del profilo, sarebbe interessante consentire il caricamento di immagini dal proprio dispositivo, con opportuni controlli su formato e dimensioni, così da offrire agli utenti una rappresentazione più fedele della propria identità.

Infine, per migliorare l'usabilità della messaggistica, si propone di introdurre strumenti di ricerca e filtraggio dei messaggi in Inbox e Outbox, ad esempio per mittente, destinatario o parole chiave, in modo da rendere più agevole il recupero di conversazioni precedenti.